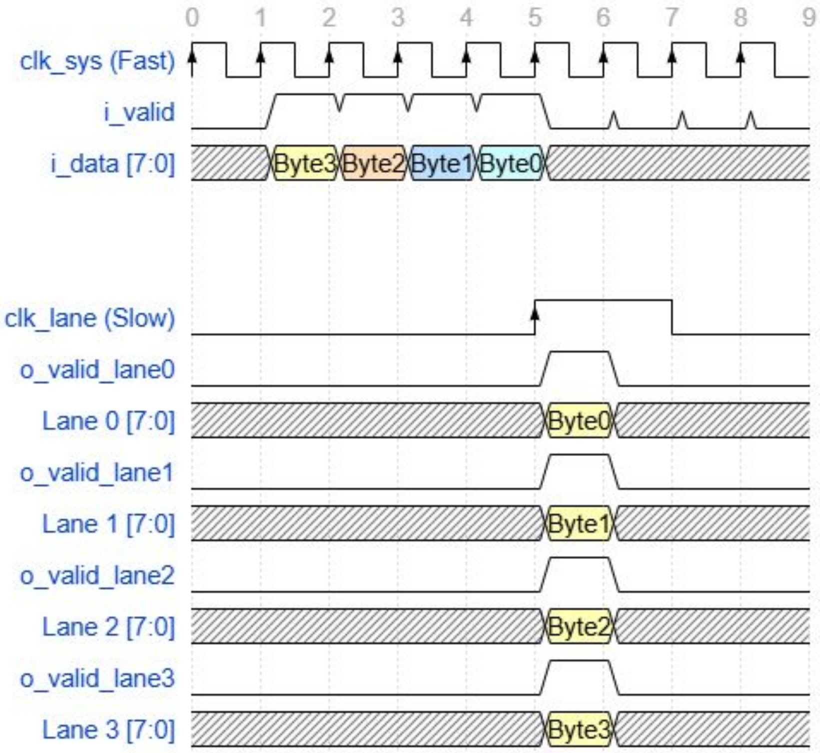
Design and Implementation of CSI Controller IP

Lane Distributor


0
1
2
3
4
5
6
7
8
9
clk_sys (Fast)
i_valid
i_data [7:0]
Byte3
Byte2
Byte1
Byte0
clk_lane (Slow)
o_valid_lane0
Lane 0 [7:0]
Byte0
o_valid_lane1
Lane 1 [7:0]
Byte1
o_valid_lane2
Lane 2 [7:0]
Byte2
o_valid_lane3
Lane 3 [7:0]
Byte3


clk
rst_n
Lane_Mode [1:0]
Data_In [7:0]
Data_Valid_In
eot
Lane Distributor
Data_Out_Lane0 [7:0]
Valid_Out_Lane0
Data_Out_Lane1 [7:0]
Valid_Out_Lane1
Data_Out_Lane2 [7:0]
Valid_Out_Lane2
Data_Out_Lane3 [7:0]
Valid_Out_Lane3

Rates Calculations

The CSI-2 serial interface shows the following features:

- Standard: MIPI CSI2 v1.3
- Input rate: 2 data lanes @ 2.5 Gbps/lane
- Pixel rate: if RGB888: up to 200 Mpixel/s, if RawBayer 10: up to 333 Mpixel/s
- Pixel format: any MIPI CSI2 v1.3: RGB565, 888, YUV422, RawBayer
- Miscellaneous: interlaced video, interleaved packets, virtual channels

If target rate = 2.5 Gbps

$$\text{TxByteClkHS} = \frac{2500\ Mbps}{8} = 312.5\ MHz$$

$$\text{System\_clk} = 312.5\ MHz\ * 4 = 1250\ MHz$$

If target rate = 891 Mbps

$$\text{TxByteClkHS} = \frac{891\ Mbps}{8} = 111.375\ MHz$$

$$\text{System\_clk} = 111.375\ MHz\ * 4 = 445.5\ MHz$$

Clock Domains

Bandwidth and Data Rate Calculation

Pixel Clock

If the video format is standard, the pixel clock frequency can be obtained from the SMPTE/CEA or VESA standards. You can also calculate the pixel clock frequency using the following equation:

$$\text{Pixel Clock}\,(\text{Hz}) = H_{TOT} * V_{TOT} * \text{FPS}$$

Where:

- H_{TOT} = total line width = active pixels + HSYNC + HFP + HBP (pixels)
- V_{TOT} = total frame height = active lines + VSYNC + VFP + VBP (lines)
- FPS = frames per second

Bandwidth and Data Rate Calculation

Total Data Rate or Bandwidth

The bandwidth of a given video format is simply a product of the Pixel Clock Frequency and Pixel Size in bits.

$$\text{Bandwidth (bps)} = \text{Pixel Clock} * \text{Bits Per Pixel}$$

Data Rate per Lane

Data Rate Per Lane (DRPL) is the bandwidth divided by the number of data lanes in the system design. CSI-2 can support up to four data lanes.

$$\text{Data Rate Per Lane (bps)} = \text{Bandwidth} \,/\, \text{number of data lanes}$$

Bandwidth and Data Rate Calculation

Example 1: 1920x1080p@60Hz, RAW10, 2-lane

Total Horizontal Samples = 2200, Total Vertical Lines = 1125

Pixel Clock Frequency = 2200 x 1125 x 60 = 148.5 MHz

Bandwidth (Total Data Rate) = 148.5 MHz * 10-bit = 1.485 Gbps

Line Rate (Data Rate per Lane) = 1.485 Gbps/2-lane = 742.5 Mbps

Example 2: 3840x2160@30Hz, RAW8, 4-lane

Total Horizontal Samples = 4400, Total Vertical Lines = 2250

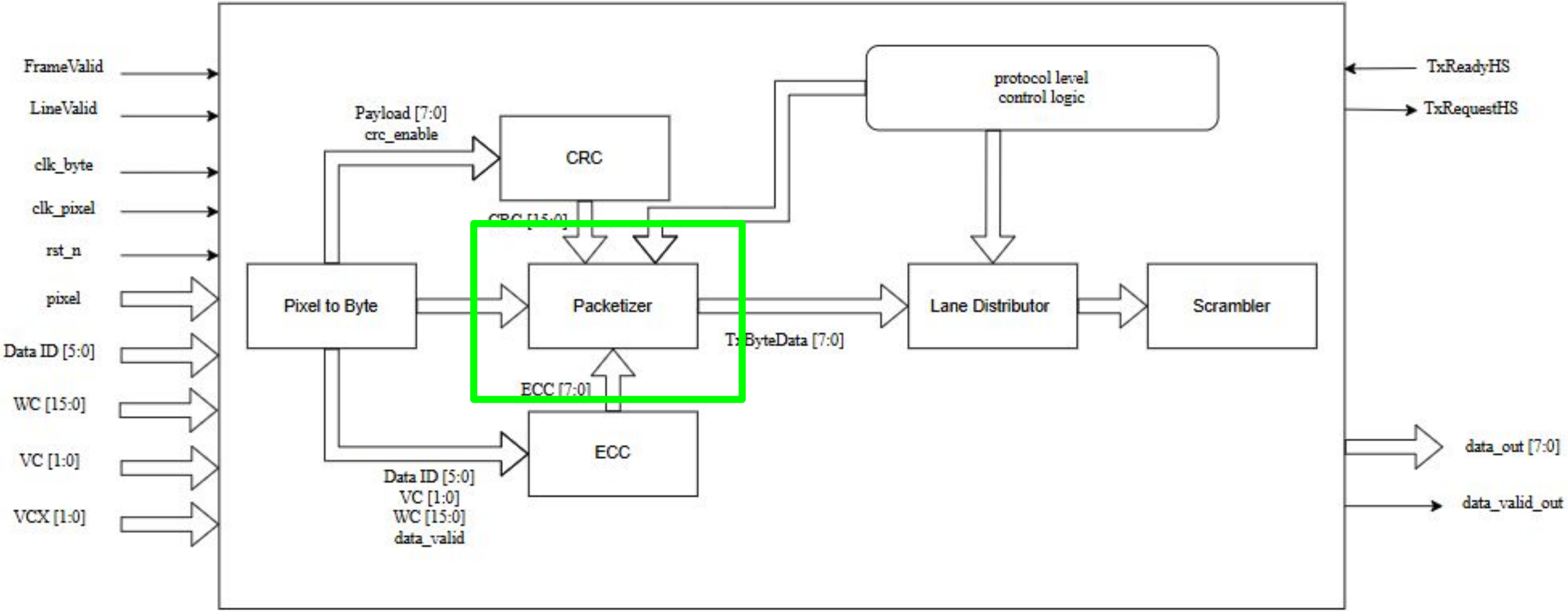
Pixel Clock Frequency = 4400 x 2250 * 30 = 297 MHz

Bandwidth (Total Data Rate) = 297 MHz * 8-bit = 2.376 Gbps

Line Rate (Data Rate per Lane) = 2.376 Gbps/4-lane = 594 Mbps

Packetizer (low Level Protocol)

Packetizer


FrameValid
LineValid
clk_byte
clk_pixel
rst_n
pixel
Data ID [5:0]
WC [15:0]
VC [1:0]
VCX [1:0]
Payload [7:0]
crc_enable
CRC
CRC [15:0]
protocol level
control logic
TxReadyHS
TxRequestHS
Pixel to Byte
Packetizer
Lane Distributor
Scrambler
TxByteData [7:0]
ECC [7:0]
ECC
Data ID [5:0]
VC [1:0]
WC [15:0]
data_valid
data_out [7:0]
data_valid_out

Packetizer Overview

Role:

- Acts as the bridge between the **Pixel-to-Byte Converter** and the the **Lane Distributor**.
- Wraps raw pixel data into legal CSI-2 packets (Long Packets).
- Generates synchronization Short Packets (Frame Start/End, Line Start/End).

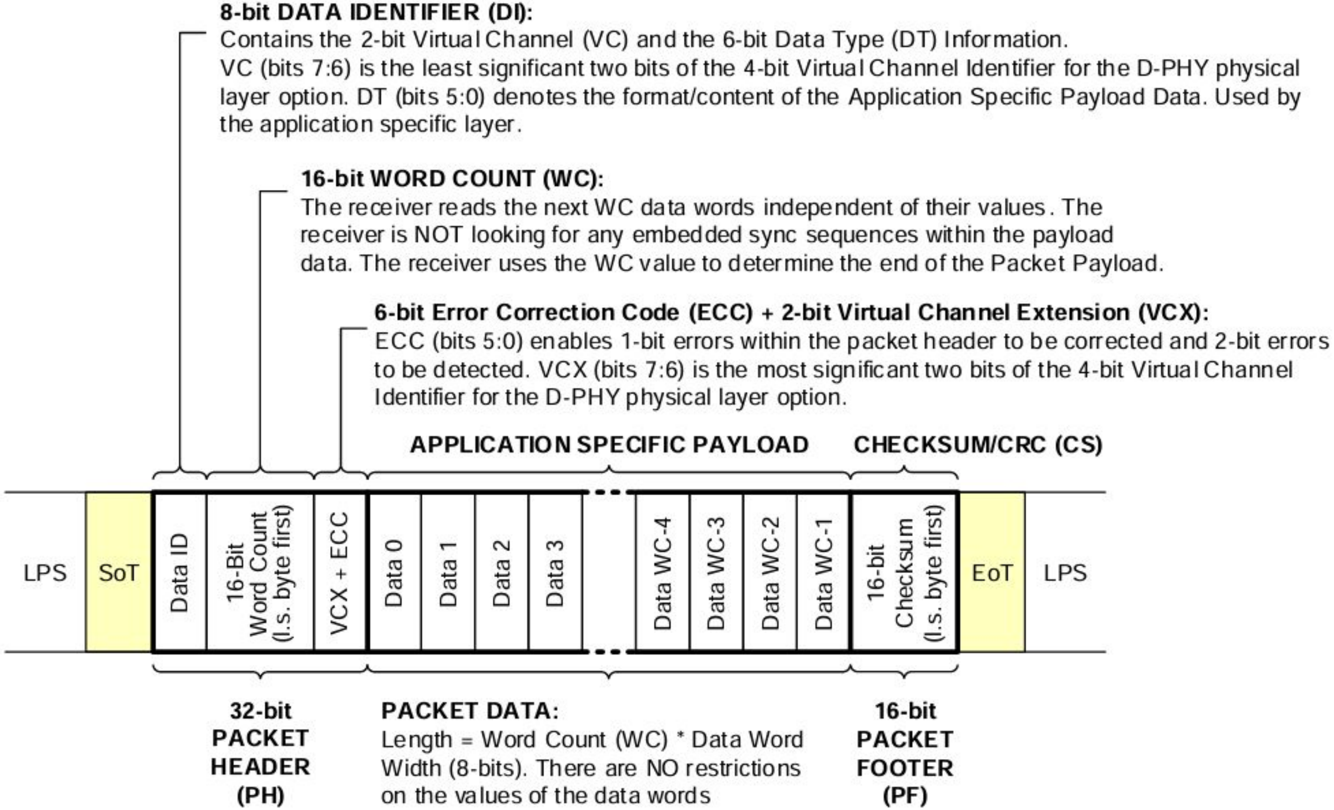
Key Responsibilities:

- **Protocol Layer:** Generates the Byte Stream (Header + Payload + Footer).
- **Header Generation:** Calculates Data ID, Word Count, and ECC.
- **Payload Alignment:** Manages the 5-cycle latency to align data with headers.
- **Footer Generation:** Calculates and appends the 16-bit CRC.

Output:

- A valid, single-stream CSI-2 byte sequence ready for **Lane Distribution**.

Long Packet Format



Short Packet Format

For Frame Synchronization Data Types the Short Packet Data Field shall be the frame number.
For Line Synchronization Data Types the Short Packet Data Field shall be the line number.
Helps the receiver keep track of which frame is being transmitted

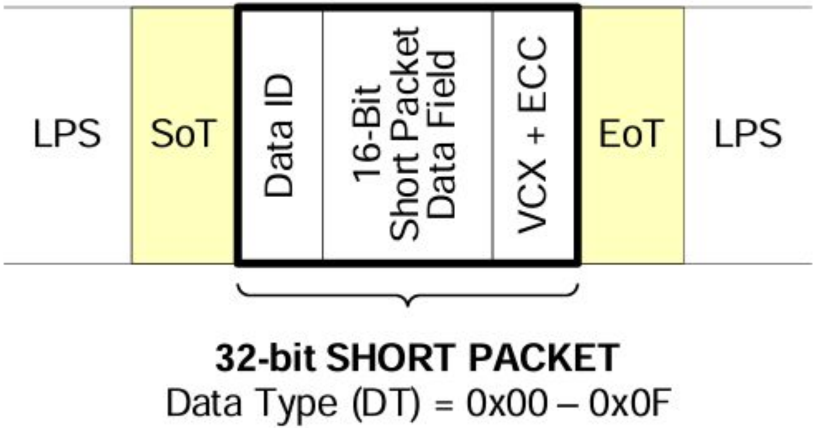


Table 13 Synchronization Short Packet Data Type Codes

Data Type	Description
0x00	Frame Start Code
0x01	Frame End Code
0x02	Line Start Code (Optional)
0x03	Line End Code (Optional)
0x04	End of Transmission Code (Optional) See ***Section 9.11.1.2.5*** for description.
0x05 to 0x07	Reserved

High-Level Architecture

The Challenge:

- Generating a Packet Header takes **4 clock cycles**.
- The raw pixel data arrives immediately when O_DATA_EN goes High.
- We cannot pause the incoming data.

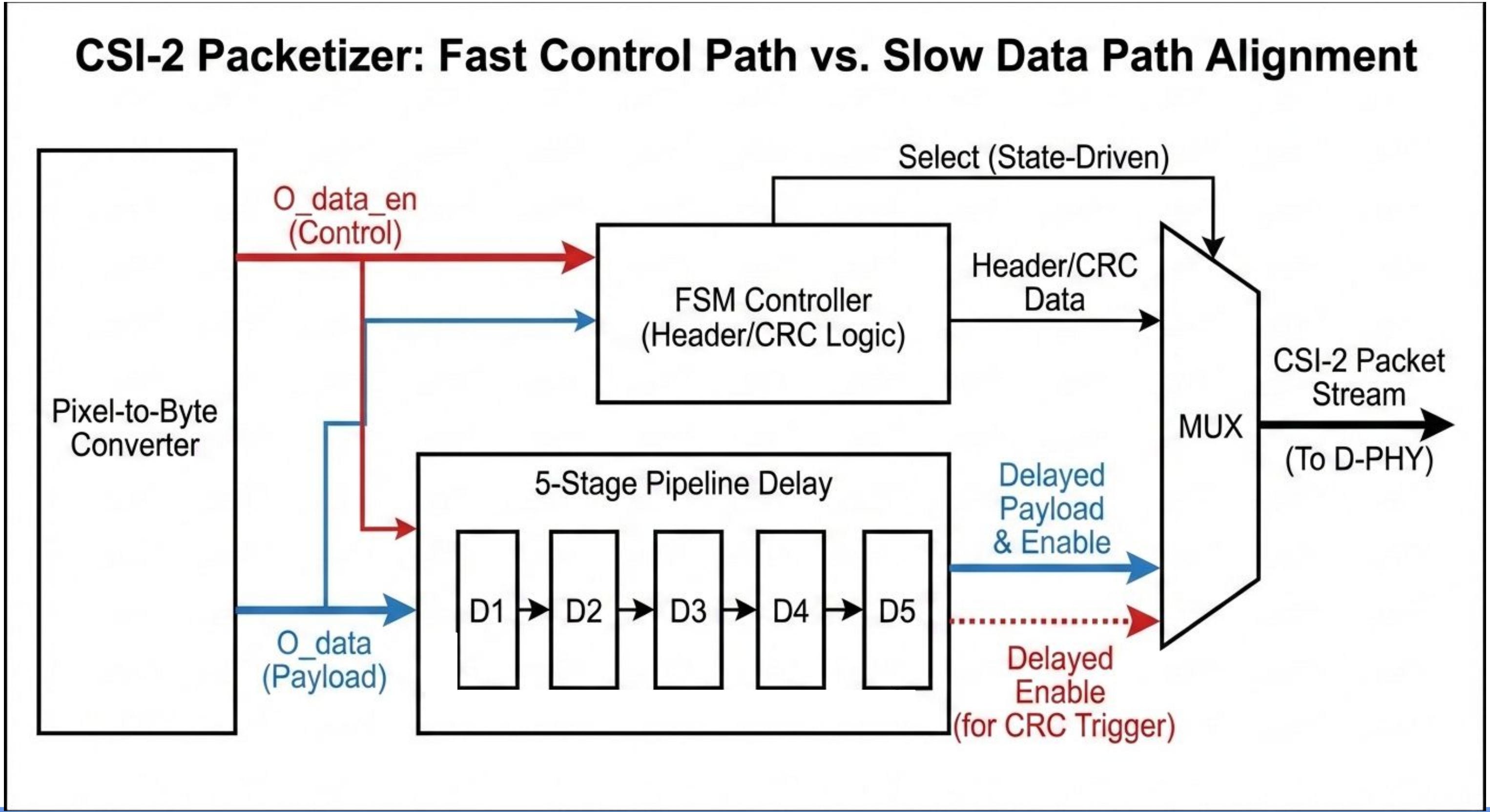
The Solution: A Dual-Path Pipeline.

- **Fast Path (Control):** The O_DATA_EN signal goes directly to the FSM to trigger the "Wake Up" and Header generation immediately.
- **Slow Path (Data):** The Payload and its Enable signal are passed through a **5-Stage Delay Line**.

Result:

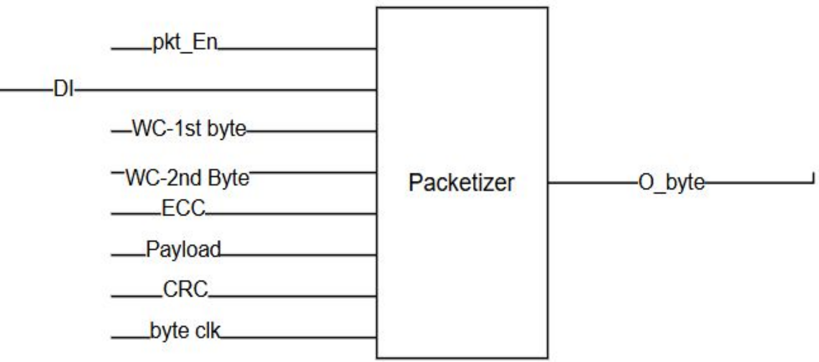
- The Payload arrives at the output MUX exactly when the Header finishes.

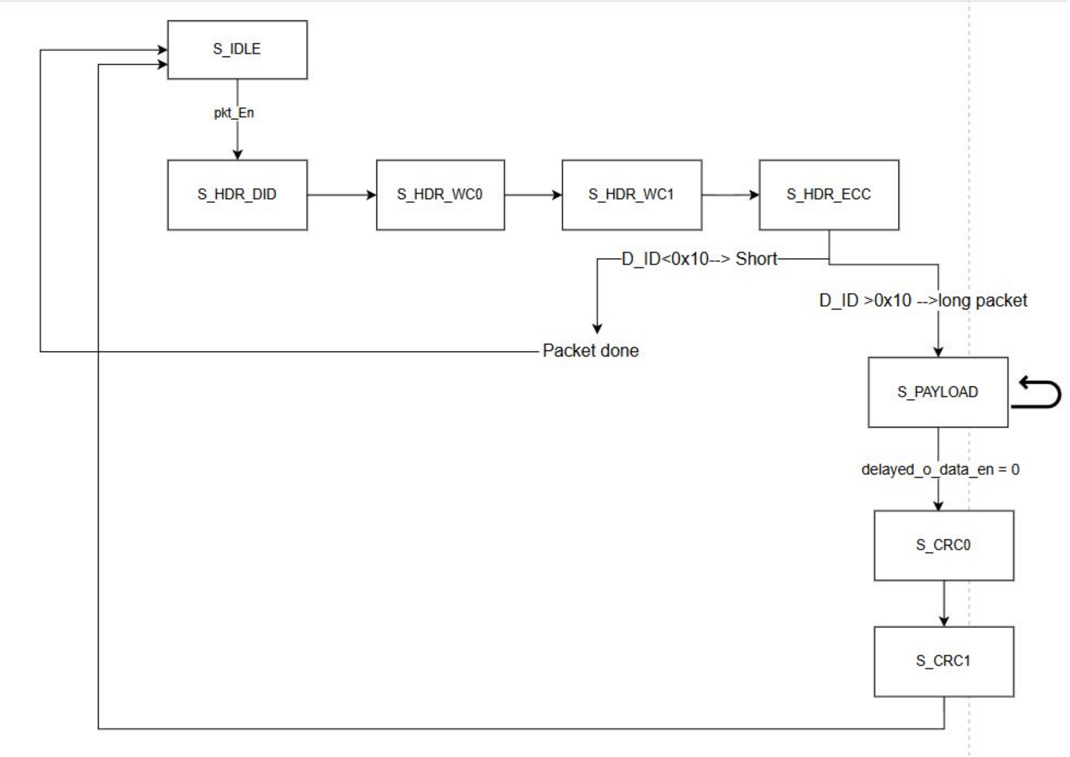
High-Level Architecture



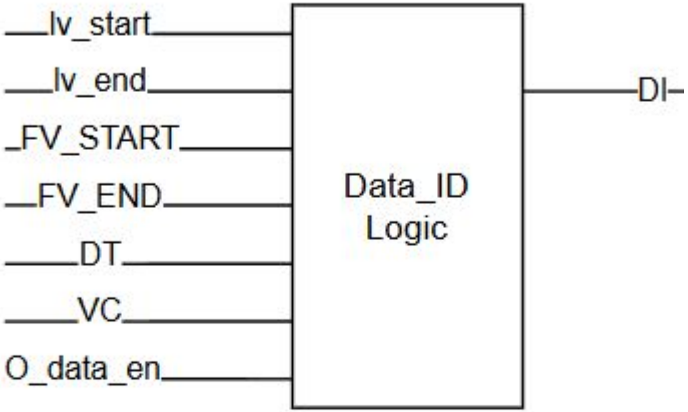
Finite State Machine:

pkt_En = FV_start |FV_end|LV_start|LV_end |O_data_en





Data ID:



Signal State	Meaning	Data ID Output	Packet Type
LV_START (Edge)	Start of Line Sync	0x02 (Line Start)	Short
O_DATA_EN (High)	Valid Image Bytes	0x2B (e.g., RAW10)	Long
LV_END (Edge)	End of Line Sync	0x03 (Line End)	Short
FV_START (Edge)	Frame Sync	0x00 (Frame Start)	Short
FV_END (Edge)	Frame Sync	0x01 (Frame End)	Short

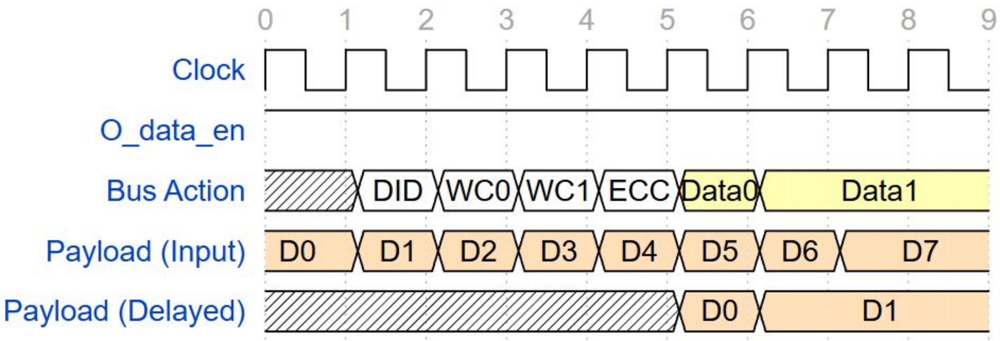
Critical Timing Constraints

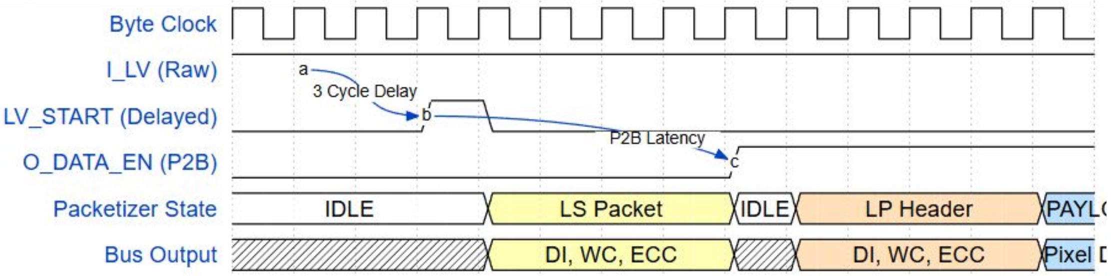
The delay between the *Pixel-to-Byte* output (O_DATA_EN) and the *Packetizer* payload input **must be exactly 5 cycles**.

Cycle 1: FSM Wakeup.

Cycles 2-5: Header Transmission (DID, WC_L, W ECC).

Cycle 6: Payload must be ready.





PPI

CSI-2 Transmitter (TX) Flow

The TX controller is the Master. It initiates all high-speed transactions.

Step 1: Initialization (ppi)

- The system starts up. The Controller ensures the D-PHY is powered up and released from reset.
- Both the **Clock Lane** and **Data Lanes** are in the **Stop State (LP-11)**. You can verify this by checking **stop_state_ck** and **stop_state_d** are **1**.

Step 2: Activate High-Speed Clock

- The Controller asserts **request_hs_ck = 1**.
- The D-PHY Clock Lane performs the LP —-> HS sequence.
- The D-PHY starts toggling the high-speed DDR clock.
- When the clock is stable, the D-PHY asserts **tx_ready_hs_ck = 1**.
- ***Note**:* Once the clock is ready, you usually keep **request_hs_ck** high for the entire duration of your video frame or burst of packets.

CSI-2 Transmitter (TX) Flow

Step 3: Request High-Speed Data Transmission

- When the Controller has a packet (Header + Payload + Footer) ready in its buffer:
- The Controller asserts **tx_request_hs_d = 1** (for all active data lanes).
- The D-PHY Data Lanes perform the SoT sequence (LP-11 —-> LP-01 —-> LP-00 —-> HS-0 —-> Sync Pattern **0xB8**).
- The D-PHY asserts **tx_ready_hs_d = 1**. This indicates the lane is now in High-Speed mode and waiting for payload bytes.

Step 4: Packet Transmission

- Upon seeing **tx_ready_hs_d** go High:
- The Controller sets **tx_valid_hs_d = 1**.
- The Controller drives the first byte of the packet (e.g., Data ID) onto **tx_data_hs_d**.

CSI-2 Transmitter (TX) Flow

Step 5: End of Transmission

- After the last byte (CRC MSB) is driven:
- The Controller de-asserts **tx_request_hs_d = 0**.
- The D-PHY automatically appends the EoT (End of Transmission) pattern and switches the Data Lanes back to Low Power mode (LP-11).
- The D-PHY de-asserts **tx_ready_hs_d**.
- The Controller waits for **stop_state_d = 1** before attempting to send the next packet.

CSI-2 Receiver (RX) Flow

The RX controller is the Slave. It must passively monitor the bus and capture data when it arrives.

Step 1: Idle State

- The Data Lanes are in the **Stop State (LP-11)**.
- The signal **RxActiveHS** is **0**.
- The Controller logic is idle, waiting for activity.

Step 2: Detection of High-Speed Entry

- The D-PHY detects the line changing from (LP) to (HS).
- The D-PHY asserts **RxActiveHS = 1**.
- The Controller's state machine moves to a "Wait for Sync" state.

Step 3: Synchronization (SoT)

- The D-PHY searches the incoming bit stream for the Start-of-Transmission (SoT) leader sequence (**0xB8**).
- **Success:** The D-PHY asserts **RxSyncHS** for one clock cycle. This tells the Controller that the byte boundaries are now aligned.

CSI-2 Receiver (RX) Flow

Step 4: Data Capture

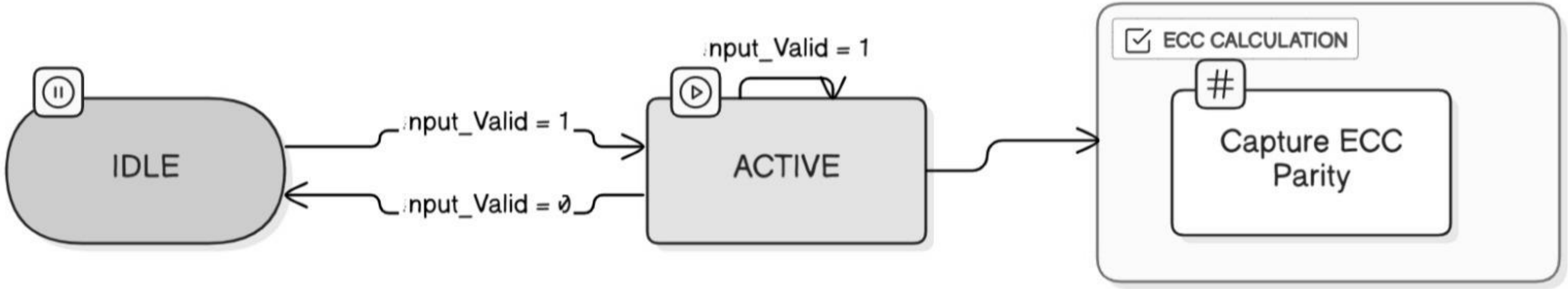
- After **RxSyncHS**, the D-PHY starts pushing deserialized bytes onto **RxDataHS**.
- The D-PHY asserts **RxValidHS = 1** whenever valid payload data is present on the bus.
- **Controller Action:** On every rising edge of **RxByteClkHS** where **RxValidHS** is **1**:
    - Read the byte from **RxDataHS**.
    - Push the byte into the RX Elasticity FIFO.

Step 5: End of Reception

- The D-PHY detects the End of Transmission (EoT) pattern or the line returning to LP-11.
- The D-PHY de-asserts **RxActiveHS = 0**.
- The Controller detects the falling edge of **RxActiveHS**. This indicates the packet is complete.
- The Controller finishes processing the packet (checks final CRC) and returns to the **Idle State**, waiting for the next burst.

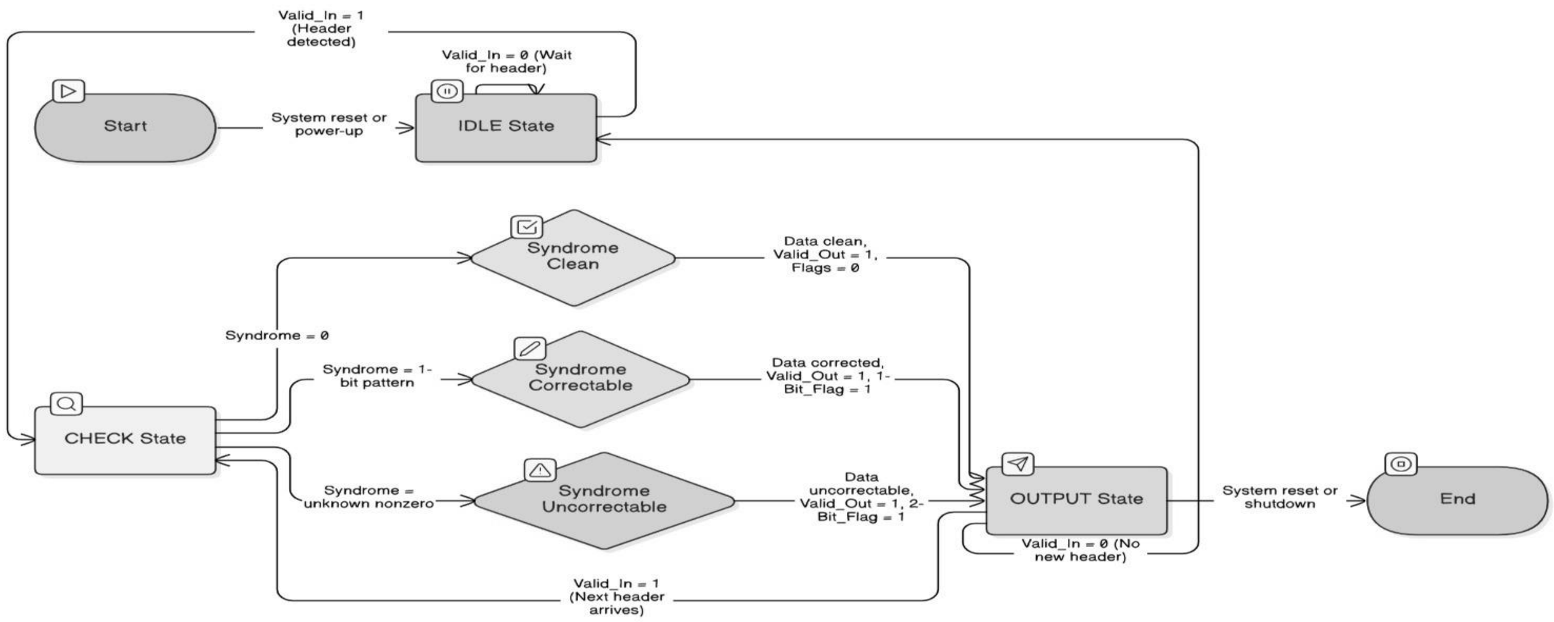
ECC

FSM (ECC TX)



IDLE
nput_Valid = 1
nput_Valid = 0
nput_Valid = 1
ACTIVE
ECC CALCULATION
#
Capture ECC Parity

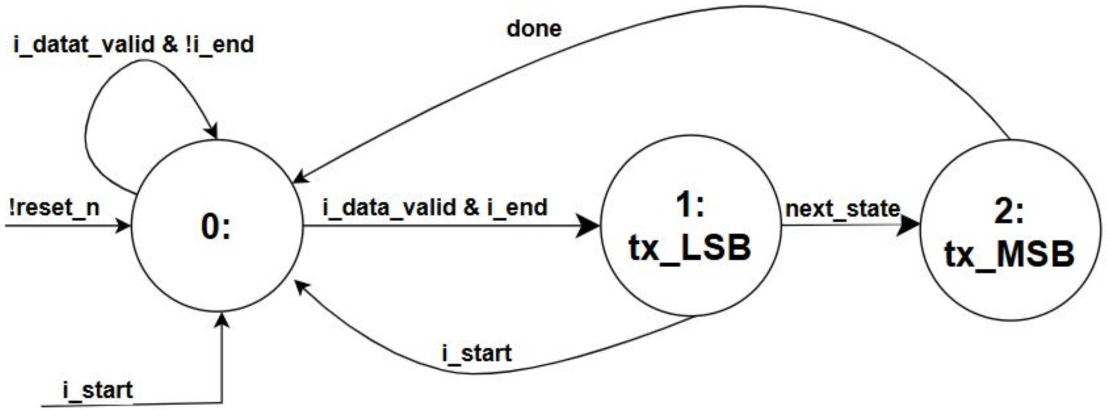
FSM (ECC RX)


Valid_In = 1 (Header detected)
Valid_In = 0 (Wait for header)
Start
System reset or power-up
IDLE State
Syndrome Clean
Data clean, Valid_Out = 1, Flags = 0
Syndrome = 0
Syndrome = 1-bit pattern
Syndrome Correctable
Data corrected, Valid_Out = 1, 1-Bit_Flag = 1
CHECK State
Syndrome = unknown nonzero
Syndrome Uncorrectable
Data uncorrectable, Valid_Out = 1, 2-Bit_Flag = 1
OUTPUT State
System reset or shutdown
End
Valid_In = 0 (No new header)
Valid_In = 1 (Next header arrives)

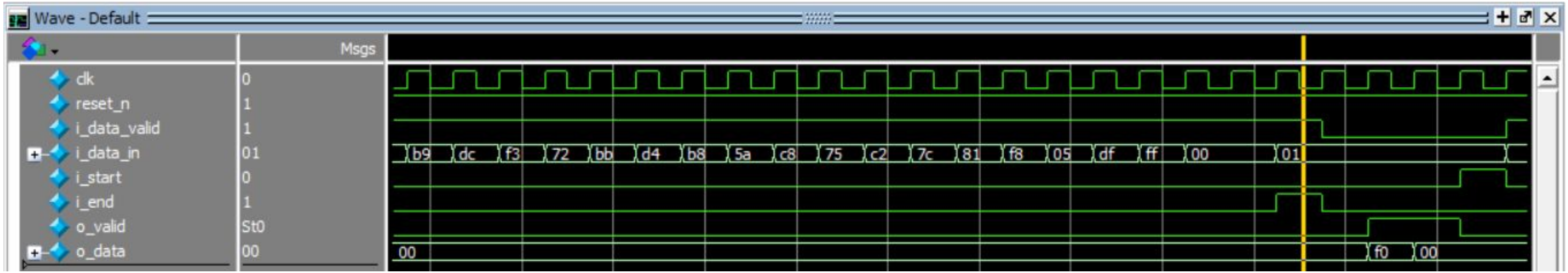
CRC

FSM

```
module crc_tx (
    input           clk,
    input           reset_n,

    input           i_data_valid,
    input   [7:0]   i_data_in,

    input           i_start,
    input           i_end,

    output reg          o_valid,
    output reg [7:0]    o_data
);
```



Simulation



Wave - Default
Msgs
clk
reset_n
i_data_valid
i_data_in
i_start
i_end
o_valid
o_data
0
1
1
01
0
1
St0
00
b9
dc
f3
72
bb
d4
b8
5a
c8
75
c2
7c
81
f8
05
df
ff
00
01
00
f0
00

Simulation

Wave - Default
Msgs
clk 0
reset_n 1
i_data_valid 0
i_data_in 00
i_start 0
i_end 0
o_crc_ok St1
o_crc_done St1
00 01 f0 01 00 ff 00 1e f0 1e c7 4f 82 78 c5 82 e0 8c 70 d2 3c 78 e9 ff 00 01 69 e5 00

Bandwidth and Data Rate Calculation

Example 1: 1920x1080p@60Hz, RAW10, 2-lane

Total Horizontal Samples = 2200, Total Vertical Lines = 1125

Pixel Clock Frequency = 2200 x 1125 x 60 = 148.5 MHz

Bandwidth (Total Data Rate) = 148.5 MHz * 10-bit = 1.485 Gbps

Line Rate (Data Rate per Lane) = 1.485 Gbps/2-lane = 742.5 Mbps

Example 2: 3840x2160@30Hz, RAW8, 4-lane

Total Horizontal Samples = 4400, Total Vertical Lines = 2250

Pixel Clock Frequency = 4400 x 2250 * 30 = 297 MHz

Bandwidth (Total Data Rate) = 297 MHz * 8-bit = 2.376 Gbps

Line Rate (Data Rate per Lane) = 2.376 Gbps/4-lane = 594 Mbps

References

References

[1] MIPI Alliance, "Specification for Camera Serial Interface 2 (CSI-2®) Version 4.0.1," MIPI Alliance, May 23, 2022

[2] MIPI Alliance, "Specification for D-PHY Version 2.5," MIPI Alliance, Jul. 5, 2019. [Online].

[3] Gowin MIPI Byte to Pixel Converter IP |Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2753/document/6820dee2cd163.pdf

[4] Intel Corporation, *“MIPI CSI-2 Intel® FPGA IP User Guide,”* Doc. No. 813926, Version 1.1.0, Apr. 26, 2024. Online. Available: https://www.intel.com/content/www/us/en/docs/programmable/813926.html

[5] Gowin MIPI Pixel to Byte Converter IP |Sep.5,2025:https://www.gowinsemi.com/upload/database_doc/2752/document/6820df0ae4095.pdf

Thank You!